

Normative Support Layer

Source State Qualification and Runtime Reliance v0.1

AGL - Qualification-to-Reliance Discipline for Long-Lived Systems

Status	Draft v0.1
Layer	Normative support layer / runtime reliance discipline
Canonical home	ester-reality-bound
Parent layer	Actor Grounding Layer v0.1.md
Author	Kotov Ivan
Location	Brussels
Year	2026

Kotov Ivan
Bruxelles, 2026

Contents

Abstract	2
1. Purpose	2
2. Scope	2
3. Non-goals	3
4. Core distinction	3
5. Qualification is not binary	4
6. Qualification dimensions	5
7. Runtime reliance classes	5
8. Mapping qualification to runtime reliance	6
9. Runtime reliance is not truth	7
10. No score soup rule	7
11. Human source qualification	7
12. Local entity qualification	8
13. Sensor and perception qualification	8
14. Delegated and proxy-origin qualification	9
15. Review is downstream of reliance	9
16. Revalidation at commit	9
17. Relationship to ARL standing and evidence	10
18. Relationship to L4 Witness	10
19. Relationship to L4 Hardware Perimeter	10
20. Relationship to ECC implementation	11
21. Examples	11
22. Fail-closed rules	12
23. Explicit bridge	12
24. Hidden bridges	12
25. Earth paragraph	13
26. Conclusion	13

Abstract

A system may correctly observe a source without being justified in **relying** on that source strongly enough to move the runtime.

That distinction matters.

Modern systems often collapse several different questions into one:

- Is the source visible?
- Is the source formatted correctly?
- Is the source admissible as evidence?
- Is the source allowed to influence runtime now?
- Is the source strong enough to authorize action or commit?

These are not the same question.

This document defines the discipline that sits inside the **Actor Grounding Layer (AGL)** between:

- qualifying the condition of a source in the present execution moment,
- and determining what degree of runtime reliance, if any, is lawful.

Its purpose is simple:

a source may be real enough to record, inspect, or review while still being too ungrounded to authorize progression, escalation, or action.

This document therefore separates:

- **source-state qualification** from
- **runtime reliance**

and prevents the system from treating visibility, coherence, or mere availability as if they automatically implied operational force.

1. Purpose

To define:

- what source-state qualification means in operational terms,
- what runtime reliance means in operational terms,
- why qualification and reliance must remain separate,
- which runtime permissions may follow from which qualification states,
- how a system prevents ungrounded sources from acquiring action force,
- how revalidation works when time passes between intake and commit,
- and how this layer supports ARL, L4 Witness, and hardware perimeter controls without collapsing into them.

2. Scope

This document applies to long-lived or bounded-action systems that must decide whether a source may:

- be recorded,

- be displayed,
- support pre-admissibility hold,
- support review admission,
- request escalation,
- narrow or widen privilege,
- influence bounded execution,
- or authorize commit-time consequence.

It applies across the four source classes defined by AGL:

- human / operator state,
- local entity initiation state,
- sensor / perception state,
- delegated / proxy / external origin state.

It applies before and during:

- action initiation,
- hold and freeze entry,
- review routing,
- oracle / remote witness request,
- re-entry decision,
- and any transition where runtime consequence may bind.

3. Non-goals

This document does **not**:

- restate the full AGL core,
- restate ARL standing or evidence doctrine in full,
- replace L4 Witness,
- replace hardware perimeter constraints,
- reduce grounding to one magical scalar score,
- or claim that qualification alone establishes truth.

It also does not equate runtime reliance with moral legitimacy. It addresses something narrower:

how much operational weight a source may lawfully carry **now**.

4. Core distinction

4.1 Source-state qualification

Source-state qualification answers:

What is the condition of this source at the present execution boundary?

This concerns:

- freshness,
- stability,
- perceptual integrity,

- cognitive load,
- authority continuity,
- physical anchoring,
- and contextual grounding.

Qualification is about **state**.

4.2 Runtime reliance

Runtime reliance answers:

Given that state, what is this source allowed to do to the system?

This may range from:

- record-only relevance,
- review-support relevance,
- escalation relevance,
- narrowed action relevance,
- or no lawful runtime force at all.

Reliance is about **permitted force**.

4.3 Why this separation matters

A source may be:

- real enough to archive,
- coherent enough to display,
- admissible enough to review,
- and still too degraded to justify immediate progression.

If qualification and reliance collapse into one, systems drift toward a dangerous shortcut:

“I can see it, therefore I can trust it strongly enough to move.”

That shortcut is one of the quiet ways fluent systems turn abstraction into action.

5. Qualification is not binary

A source is rarely just “good” or “bad”.

Its grounding may be strong in one dimension and weak in another.

A human operator may still be authenticated but cognitively overloaded.

A local entity may still be continuous but temporarily drifted.

A sensor path may still be live but stale relative to the decision window.

A delegated source may still be structurally well-formed but too abstracted from the live execution boundary.

That is why qualification should remain:

- multi-dimensional,
 - bounded,
 - and fail-closed under ambiguity.
-

6. Qualification dimensions

The following dimensions should be considered when determining source-state qualification.

6.1 Physical grounding

Is the source tied to the live operational perimeter strongly enough to matter now?

6.2 Temporal grounding

Is the source fresh enough relative to the present execution boundary?

6.3 Cognitive grounding

Is the source stable enough in attention, overload, fatigue, or interpretive integrity?

6.4 Perceptual grounding

Is the acquisition path trustworthy enough for runtime use?

6.5 Authority grounding

Does the source still hold lawful privilege now, not merely earlier?

6.6 Continuity grounding

Is the source part of the correct continuity-bearing line?

6.7 Environmental grounding

Do current L4 conditions still support reliance on this source?

These dimensions do not all need equal weight every time. But they must not be flattened into decorative vocabulary.

7. Runtime reliance classes

Runtime reliance should be expressed as a bounded set of operational classes.

A source may be strong enough for one class and too weak for the next.

7.1 RECORD_ONLY

The source may be preserved as trace or history, but has no action force.

Typical use: - ambiguous signal, - degraded source, - archive without runtime influence.

7.2 VISIBLE_BUT_NON_AUTHORITATIVE

The source may be shown, inspected, or compared, but not used to authorize runtime progression.

Typical use: - visibility without legitimacy, - graph/display surfaces, - unresolved branches.

7.3 REVIEW_SUPPORT_ONLY

The source may help justify hold, freeze, or review, but not direct execution or release.

Typical use: - disputed but meaningful input, - bounded review support.

7.4 ESCALATION_ELIGIBLE

The source may request bounded escalation, such as oracle-assisted review or stronger contradiction handling, but still does not authorize ordinary action on its own.

7.5 ACTION_ELIGIBLE_WITH_LIMITS

The source may support bounded action, but only under narrowed privilege, stricter logging, or tighter time windows.

7.6 COMMIT_ELIGIBLE

The source is grounded enough to support action whose consequence may bind, subject to any required revalidation at commit.

7.7 RUNTIME_RELIANCE_DENIED

The source may not lawfully influence runtime progression under current conditions.

This is stronger than caution. It is a hard refusal of operational force.

8. Mapping qualification to runtime reliance

Qualification state and runtime reliance are related but not identical.

Recommended mapping:

8.1 GROUNDED

May support: - ACTION_ELIGIBLE_WITH_LIMITS - or COMMIT_ELIGIBLE depending on privilege and scope.

8.2 GROUNDED_WITH_CAUTION

May support: - REVIEW_SUPPORT_ONLY - ESCALATION_ELIGIBLE - or bounded ACTION_ELIGIBLE_WITH_LIMITS

Default posture: narrow privilege, widen logging, shorten window.

8.3 HOLD_REQUIRED

May support: - RECORD_ONLY - VISIBLE_BUT_NON_AUTHORITATIVE - REVIEW_SUPPORT_ONLY

It must not silently promote itself into ordinary action.

8.4 QUARANTINE_REQUIRED

May support: - RECORD_ONLY - limited inspection, - bounded review handling

It must not support normal re-entry or privileged continuation.

8.5 RUNTIME_RELIANCE_DENIED

Supports: - trace preservation only, - and possibly witness-bound refusal.

No escalation, no ordinary action, no quiet fallback.

8.6 REVALIDATE_AT_COMMIT

The source may support provisional routing or bounded intermediate steps, but must be checked again before consequence binds.

9. Runtime reliance is not truth

A source with high runtime reliance is not thereby “true”. A source with denied runtime reliance is not thereby “false”.

This layer does not certify ontology. It certifies **how strongly the runtime may depend on the source now**.

That difference matters because many systems confuse:

- “I cannot safely rely on this now” with
- “this never mattered”

Those are not the same conclusion.

A mature system preserves that distinction.

10. No score soup rule

This layer must resist a recurring implementation temptation:

- give every source a floating-point score,
- average dimensions together,
- call the result “grounding,”
- and let a threshold decide everything.

That is convenient. It is also structurally sloppy.

Different dimensions carry different failure implications. For example:

- weak perceptual grounding is not the same as weak authority grounding,
- weak cognitive grounding is not the same as weak continuity grounding,
- stale timing is not the same as broken physical perimeter.

Systems may still use internal scoring for helper logic, but the exposed runtime decision should remain:

- typed,
 - bounded,
 - and interpretable by class, not hidden inside a blended numeric fog.
-

11. Human source qualification

A human source may remain:

- the owner,
- the anchor,
- the responsible party,

and still be too degraded **in this moment** to support ordinary runtime reliance.

This includes cases of:

- fatigue,
- sleep deprivation,
- overload,
- panic,

- stress compression,
- physiological impairment,
- and unstable attention under consequence.

The purpose here is not paternalism. It is the prevention of bad commits under bad human state.

A serious system protects human agency by refusing to convert every immediate human state into full execution authority.

12. Local entity qualification

A persistent local entity may still be too degraded to support ordinary reliance.

This may arise from:

- unresolved drift,
- continuity fracture,
- overloaded review loops,
- local anomaly state,
- unresolved deadlock,
- or narrowed confidence under L4 pressure.

Continuity alone does not equal present grounding. A long-lived system must be able to say:

 this is still the same entity,
 but it is not in a good enough state to authorize ordinary progression right
 now.

That sentence is painful. It is also necessary.

13. Sensor and perception qualification

A live sensor is not automatically a grounded sensor.

Perceptual paths may fail through:

- staleness,
- low integrity,
- partial detachment,
- missing calibration,
- environmental corruption,
- or broken acquisition chain.

A system that treats all available perception as action-worthy perception invites confident error under noise.

That is why perceptual grounding should decide not only: - whether something is seen, but - how much runtime force the seeing is allowed to carry.

14. Delegated and proxy-origin qualification

Modern systems frequently act through delegation, proxying, provider mediation, and abstracted surfaces.

That is operationally useful. It is also where grounding often thins out.

A delegated source may still be:

- structurally authenticated,
- policy-shaped,
- and contextually too remote from the live execution edge to justify strong reliance.

This is exactly why a system must not say:

proxy exists, therefore runtime trust exists.

Delegation may preserve reach while degrading grounding. The layer must see that.

15. Review is downstream of reliance

A source does not need full action eligibility to matter.

It may still be strong enough to:

- justify a pre-admissibility hold,
- support freeze,
- route into review,
- or open contradiction handling.

That is why REVIEW_SUPPORT_ONLY exists.

This is also why AGL is not identical to ARL.

ARL asks: - who has standing, - what evidence is admissible, - what outcome is lawful.

This document asks: - what force may this source carry before and during those steps?

The order matters.

16. Revalidation at commit

A source may be grounded enough at intake and ungrounded enough at commit.

This is common when:

- delay is long,
- environmental posture changes,
- the human source deteriorates,
- the local entity drifts,
- sensor trust decays,
- or privilege expires before consequence binds.

Therefore, high-impact action should often require commit-time revalidation of

source-state qualification.

This is not indecision. It is preventing stale legitimacy from becoming live damage.

17. Relationship to ARL standing and evidence

17.1 Standing without grounding

A claimant may have standing in principle while being too degraded now for ordinary operational reliance.

17.2 Evidence without grounding

A signal may be worthy of preservation, witness binding, or later review, while still being too ungrounded to authorize ordinary runtime dependence.

17.3 Grounding without standing

A source may be well-grounded and still lack the procedural right to move the dispute machinery.

These are distinct axes.

Systems that collapse them create avoidable confusion.

18. Relationship to L4 Witness

L4 Witness records and binds what happened.

This document governs whether a source may carry enough runtime force for action, escalation, or review to rely on it strongly in the first place.

Witness can still faithfully record a badly grounded beginning. That is why witness and qualification are neighbors, not substitutes.

A clean audit trail does not repair a bad initiation. It proves it happened.

19. Relationship to L4 Hardware Perimeter

This layer is upstream semantics for perimeter reality.

Hardware perimeter already constrains:

- radios,
- network mode,
- power path,
- maintenance posture,
- anomaly flags,
- circuit breakers,
- physical access.

This document gives those conditions a runtime reliance consequence.

Example: a source may remain formally valid, but if perimeter posture is degraded, runtime reliance may still need to drop from COMMIT_ELIGIBLE to HOLD_REQUIRED or RUNTIME_RELIANCE_DENIED.

That is exactly the sort of translation this layer exists to make.

20. Relationship to ECC implementation

Implementation-facing ARL work already contains the mechanical skeleton:

- intake hooks,
- hold-entry hooks,
- privilege hooks,
- witness hooks,
- routing hooks,
- re-entry hooks,
- dispute persistence.

This document tells those hooks what upstream question they are helping answer:
 how much operational force may this source carry now?

Without that question, the hooks still exist, but they become semantically thinner than the architecture requires.

21. Examples

21.1 Fatigued operator, lawful account

The operator is authenticated and technically authorized, but is visibly overloaded and initiating a high-impact path late in the cycle.

Qualification: GROUNDED_WITH_CAUTION or HOLD_REQUIRED

Runtime reliance: - not full commit, - possibly REVIEW_SUPPORT_ONLY - or narrow ACTION_ELIGIBLE_WITH_LIMITS

21.2 Continuous local entity after anomaly spike

The entity remains continuous, but recent anomaly flags and unresolved drift indicate unstable local initiation.

Qualification: HOLD_REQUIRED or REVALIDATE_AT_COMMIT

Runtime reliance: - route to review, - deny ordinary escalation, - maintain fail-closed posture.

21.3 Live sensor with stale timing

Perception is available, but timing is outside the valid commit window.

Qualification: temporally weak despite perceptual availability

Runtime reliance: - VISIBLE_BUT_NON_AUTHORITATIVE - perhaps REVIEW_SUPPORT_ONLY - not direct action.

21.4 Remote proxy request with valid credentials

The proxy is authenticated, but too detached from live execution state to support immediate high-impact action.

Qualification: authority structurally present, present-state grounding weak

Runtime reliance: - ESCALATION_ELIGIBLE - not COMMIT_ELIGIBLE

These examples matter because they show the central point: **qualification and**

runtime reliance are not synonyms.

22. Fail-closed rules

22.1 No strong reliance from weak grounding

A source may remain visible without remaining action-worthy.

22.2 No stale commit

Earlier legitimacy does not automatically survive delay.

22.3 No proxy laundering

Delegation must not silently convert abstraction into runtime force.

22.4 No human convenience override

The fact that a human wants immediate execution does not erase degraded human state.

22.5 No cosmetic continuity

A long-lived entity does not gain present grounding merely by being the same entity as before.

22.6 No smooth promotion of unresolved source state

If qualification is weak, the system must not polish the weakness into ordinary reliance through confidence language.

23. Explicit bridge

source-state qualification <-> bounded runtime reliance <-> ARL / witness / perimeter discipline

Qualification answers the present-state question. Runtime reliance answers the operational-force question. ARL answers the procedural-conflict question. Witness preserves what actually happened. Perimeter supplies live environmental truth.

24. Hidden bridges

24.1 Hidden bridge #1 - ERB <-> SER

This document explains why pre-admissibility hold is not merely a polite pause before review. It is the place where grounding weakness can still reduce runtime force before procedural machinery becomes too late.

24.2 Hidden bridge #2 - ERB <-> ECC

This document gives meaning to implementation hooks such as: intake interception, privilege narrowing, commit-time revalidation, and re-entry denial. Without a qualification-to-reliance layer, those hooks stop movement but do not fully explain what degree of force the source was denied.

25. Earth paragraph

In aviation, a pilot may still be licensed, the instrument panel may still be lit, and the aircraft may still be technically controllable - yet an approach can still be denied because wind shear, pilot state, instrument trust, or runway conditions reduce what the system may safely rely on **right now**.

That is not a statement that the pilot is unreal, the aircraft is false, or the instruments never mattered.

It is a statement about present operational force.

Source-state qualification and runtime reliance have the same relationship. A serious system must know the difference between: what exists, what is visible, and what may still move the machine.

26. Conclusion

This document formalizes a simple but load-bearing distinction:

a source may be sufficiently real to record, inspect, or review
while still being too weakly grounded to authorize runtime progression, escalation, or commit.

That is why source-state qualification and runtime reliance must remain separate.

Qualification determines the condition of the source. Runtime reliance determines how much operational force that condition may lawfully carry.

Without this separation, systems drift toward a familiar lie:

if I can see it, I can trust it enough to move.

For serious systems, that is not maturity. It is shortcut architecture.

This layer exists to stop that shortcut before it reaches action.